

D-CIPHER: Dynamic Collaborative Intelligent Multi-Agent System with Planner and Heterogeneous Executors for Offensive Security

Meet Udeshi*, Minghao Shao*, Haoran Xi*, Nanda Rani, Kimberly Milner, Venkata Sai Charan Putrevu, Brendan Dolan-Gavitt, Sandeep Kumar Shukla, Prashanth Krishnamurthy, Farshad Khorrami, Ramesh Karri, Muhammad Shafique

Abstract—Large Language Models (LLMs) have been used in cybersecurity such as autonomous security analysis or penetration testing. Capture the Flag (CTF) challenges serve as benchmarks to assess automated task-planning abilities of LLM agents for cybersecurity. Early attempts to apply LLMs for solving CTF challenges used single-agent systems, where feedback was restricted to a single reasoning-action loop. This approach was inadequate for complex CTF tasks. Inspired by real-world CTF competitions, where teams of experts collaborate, we introduce the D-CIPHER LLM multi-agent framework for collaborative CTF solving. D-CIPHER integrates agents with distinct roles with dynamic feedback loops to enhance reasoning on complex tasks. It introduces the *Planner-Executor agent system*, consisting of a Planner agent for overall problem-solving along with multiple heterogeneous Executor agents for individual tasks, facilitating efficient allocation of responsibilities among the agents. Additionally, D-CIPHER incorporates an *Auto-prompter agent* to improve problem-solving by auto-generating a highly relevant initial prompt. We evaluate D-CIPHER on multiple CTF benchmarks and LLM models via comprehensive studies to highlight the impact of our enhancements. Additionally, we manually map the CTFs in NYU CTF Bench to MITRE ATT&CK techniques that apply for a comprehensive evaluation of D-CIPHER’s offensive security capability. D-CIPHER achieves state-of-the-art performance on three benchmarks: 22.0% on NYU CTF Bench, 22.5% on Cybench, and 44.0% on HackTheBox, which is 2.5% to 8.5% better than previous work. D-CIPHER solves 65% more ATT&CK techniques compared to previous work, demonstrating stronger offensive capability. D-CIPHER is available at https://github.com/NYU-LLM-CTF/nyuctf_agents as the `nyuctf_multiagent` package. The MITRE ATT&CK techniques mapping is available at https://github.com/NYU-LLM-CTF/NYU_CTF_Bench under the `mitre_attack_mapping` folder.

Index Terms—Capture The Flag, Large Language Models, Multi-Agent Systems

I. INTRODUCTION

LARGE language models (LLMs) have demonstrated remarkable potential in cybersecurity applications such as vulnerability detection [2, 16, 23], bug localization [21, 50],

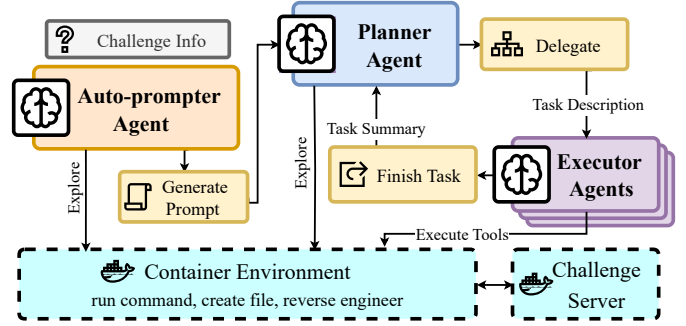


Fig. 1. Overview of D-CIPHER. The Auto-prompter, Planner, and heterogeneous Executors all collaborate and interact to solve the CTF.

and automated program repair [5, 42]. Recent advances in LLMs have led to their application to autonomously perform complex cybersecurity tasks [3, 38]. Autonomous agents for offensive security are critical to counter the rapidly expanding cyber threats [12, 13, 44]. Capture the Flag challenges (CTFs) are suitable for improving cybersecurity skills [9, 37]. CTFs help evaluate LLM proficiency in cybersecurity and automated task planning by simulating real-world offensive security scenarios [27, 29, 31, 35, 46], as they contain complex tasks requiring expertise across cryptography, digital forensics, binary exploitation, and reverse engineering. Autonomous LLM agents are evaluated with jeopardy-style CTFs involving standalone software which after successfully compromising reveals a unique “flag” string as a clear indicator of success. Offensive security capabilities of LLM agents can also be benchmarked using the MITRE ATT&CK framework [24] that offers real-world threat classification [4, 8].

Most current LLM agents for CTFs operate as single agents handling challenges end-to-end. However, CTFs are complex and require exploration and sequential task execution. Single-agent setups limit feedback to self-reflection, often leading to retries, loss of focus, and hallucinations. In contrast, real-world CTFs are team-based, involving diverse expertise [7, 10], which current frameworks fail to reflect. While multi-agent systems are gaining traction in other fields [14, 20, 43], their use in cybersecurity is still nascent. Offensively, they can automate tasks like pentesting and exploit generation [6, 32]; defensively, they aid in bug discovery and repair [19]. Motivated by this, we propose a multi-agent LLM framework

*Authors contributed equally to this research.

M. Udeshi, M. Shao, H. Xi, K. Milner, V.S.C. Putrevu, B. Dolan-Gavitt, P. Krishnamurthy, F. Khorrami, and R. Karri are with the NYU Tandon School of Engineering. M. Shao and M. Shafique are with NYU Abu Dhabi. N. Rani and S.K. Shukla are with the Indian Institute of Technology Kanpur.

This work was supported in part by the NYUAD Center for Artificial Intelligence and Robotics (CAIR), funded by Tamkeen under the NYUAD Research Institute Award CG010, NYUAD Center for Cyber Security (CCS), funded by Tamkeen under the NYUAD Research Institute Award G1104.

that assigns distinct roles to agents, enabling dynamic and collaborative problem-solving.

We present D-CIPHER, a novel LLM multi-agent framework to autonomously solve CTFs via collaboration of multiple LLM agents. D-CIPHER introduces two mechanisms for enhanced interaction and dynamic feedback between agents: first, the *Planner-Executor agent system* that involves a Planner to solve the CTF end to end, and multiple heterogeneous Executor agents to complete single tasks assigned by the Planner; and second, the *Auto-prompter agent* to explore the CTF environment and generate an initial prompt for the main system. Dividing responsibilities between planner and executors allows each agent to maintain focus for long complex tasks, and reduces hallucinations. Auto-prompting is a prompt engineering technique to improve LLM performance by generating dynamic task-specific prompts as opposed to human-written hard-coded prompt templates. D-CIPHER incorporates auto-prompting as a separate agent to produce a highly-relevant initial prompt to kick-start the main system. Additionally, we are the first work to evaluate LLM agents using the MITRE ATT&CK framework [24]. We augment the NYU CTF Bench [31] with a mapping of ATT&CK techniques to evaluate D-CIPHER and related LLM agents by the techniques they employ for the CTFs, offering a comprehensive overview of the agent’s offensive security capability.

Figure 1 shows an overview of D-CIPHER. All agents access a shared container environment to run shell commands and interact with the CTF server. The Auto-prompter starts the process by exploring the environment and generating a prompt for the Planner. The Planner also explores for a few rounds, after which it creates a plan and delegates tasks to the Executors. Each delegated task initiates a new Executor with a new conversation history, allowing for heterogeneous execution and greater focus on single tasks. After completing the task, the Executor returns a task summary which the Planner may use to update the plan and delegate further tasks. The Planner-Executor loop continues until the challenge is solved, or some terminal conditions are met. This collaborative design allows D-CIPHER to tackle complex CTFs, improving performance to achieve state-of-the-art accuracy on CTF benchmarks.

We evaluate D-CIPHER seven LLM models, via it’s accuracy on three benchmarks and it’s performance in solving MITRE ATT&CK techniques. Our results demonstrate that the multi-agent approach not only improves problem-solving, but also enhances robustness by mitigating errors and dynamically adapting strategies during runtime. We perform ablation studies and comparison with related works to further illustrate D-CIPHER’s ability to outperform single-agent systems. Performance on the MITRE ATT&CK techniques additionally reveals D-CIPHER’s superior offensive security capability. The contributions of this work are as follows:

- 1) *D-CIPHER*, a novel LLM multi-agent framework that leverages specialized agents with distinct roles to enable agent collaboration for autonomous problem-solving
- 2) A novel *Planner-Executor system* with a Planner and multiple Executors to divide responsibilities and enhance long-term focus for complex problems

- 3) A novel *Auto-prompter agent* that improves auto-prompting with an agent setup
- 4) Augmenting the NYU CTF Bench by mapping MITRE ATT&CK techniques and elaborating D-CIPHER’s offensive security capability
- 5) A comprehensive study on how multi-agent collaboration between agents enhances problem-solving on CTFs

The paper is structured as follows: Section II provides background and reviews related work, Section III describes D-CIPHER’s implementation, Section IV outlines the experimental setup, Section V presents the results, Section VI discusses common failures and ethics, and Section VII concludes the paper and proposes directions for future work.

II. RELATED WORK

Autonomous frameworks create a feedback loop to allow the LLM to perform tasks and operate as autonomous agents. LLMs are supporting function (or tool) calling where actions can be provided that the LLM may choose to “call” as a function. Many “tools” can be provided such as a command line, web search, file editing, and code execution [40]. To help LLMs on long-horizon tasks, plan-and-solve prompting [39] enhances long-term focus via a planning phase before iterative execution to tackle complex tasks [36]. ReAct (reasoning + action) [48] combines step-by-step reasoning with action. The prompting methods in these agents involve static hard-coded templates where environment and task information is filled in. These often fail to adapt to different problems. Auto-prompting [33, 51, 52] allows the LLM itself to generate a highly-relevant prompt, invoking factual responses and reducing hallucinations. D-CIPHER incorporates auto-prompting as a separate agent that can explore the environment and generate a better prompt. Expanding on single LLM agents, Multi-agent systems enhance problem-solving by collaboration between specialized agents, working on different aspects of complex tasks [15]. Multi-agent systems are effective in cybersecurity applications such as insider threat detection [34], incident response [22], and improving code safety [26].

TABLE I
FEATURE COMPARISON OF CTF SOLVING AGENTS.

Study	# CTFs	Open bench	Tool use	Autonomous	Multi-agent	Auto-prompt
Tann et al. [35]	7	✗	✗	✗	✗	✗
Shao et al. [30]	26	✗	✓	✓	✗	✗
InterCode-CTF[46]	100	✓	✓	✓	✗	✗
NYU CTF Bench [31]	200	✓	✓	✓	✗	✗
Turtayev et al. [36]	100	✓	✓	✓	✗	✗
Cybench [49]	40	✓	✓	✓	✗	✗
EnIGMA [1]	350	✓	✓	✓	✗	✗
HackSynth [25]	200	✓	✓	✓	✓	✗
D-CIPHER (ours)	290	✓	✓	✓	✓	✓

Recent works build LLM agents targeted towards CTFs. Table I shows a feature comparison of D-CIPHER with related works on LLM agents for autonomous CTF solving. The

D-CIPHER Multi-Agent System

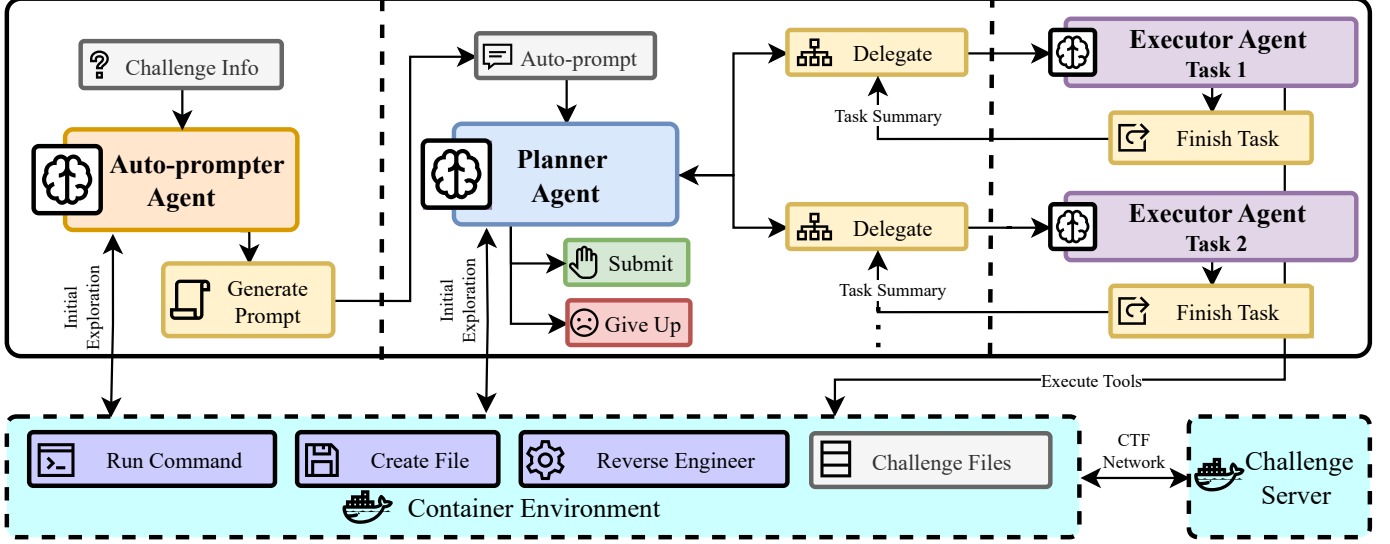


Fig. 2. Workflow of the D-CIPHER multi-agent system. Execution starts with the Auto-prompter which explores the CTF and produces a dynamic, relevant prompt. The Planner proceeds with exploration and delegates specific tasks to the Executors. Each Executor starts with a fresh conversation history to focus on the delegated task, while the Planner maintains overall context and drives the problem solving.

InterCode-CTF agent [45] reveals that LLM agents demonstrate basic cybersecurity skills but struggle with more complex tasks. The NYU CTF baseline agent [30] integrates external tools and shows improved potential of tool-assisted LLMs to solve CTFs, however the agent exhausts LLM context length when command output history grows. InterCode-CTF manages this by truncating the history to the last few iterations. Even so, agents face issues with longer tasks. Agents perform better with a focused set of tools with well-defined interfaces [47]. EnIGMA [1] agent incorporates interactive tools, in-context learning, and LLM summarizer for context management to achieve state-of-the-art results. HackSynth [25] uses iterative planning and feedback summarization stages which helps to finish multiple tasks and improves overall problem solving. Similarly, Cybench [49] introduces a benchmark of 40 CTFs augmented with step-by-step tasks, focusing LLM agents on each smaller task. Turtayev et al. [36] expand on InterCode-CTF by implementing plan-and-solve prompting, significantly improving on InterCode-CTF benchmark. These works highlight that LLM agents excel at implementing code and executing commands to accomplish small concrete tasks when provided with dynamic feedback and task-specific toolsets. While these works involved multiple LLMs with different tasks such as planning and summarizing along-side a main agent, D-CIPHER is the first work to formulate a multi-agent system for CTFs with division of responsibilities and well-defined interactions for dynamic feedback.

III. D-CIPHER IMPLEMENTATION

The D-CIPHER framework introduces a collaborative LLM multi-agent system. Each individual agent is based on the NYU CTF baseline agent [31] with upgraded prompts that describe tasks and additional interaction tools for a multi-agent collaborative context. We use function calling features

of current LLMs to prompt for agent actions. The system has three agents: (1) the *Planner agent* generates the overall plan to solve the CTF challenge, delegating specific tasks to Executors, and revising the plan based on their feedback; (2) the *Executor agent* performs the task delegated by the Planner and returns a summary; and (3) the *Auto-prompter agent* generates a dynamic prompt based on its exploration of the CTF. Figure 2 shows D-CIPHER’s workflow.

A. Context Management

Each agent maintains a conversation history of LLM inputs and outputs. An LLM agent’s context contains: (1) the system prompt that sets the agent’s role and provides actions, (2) the initial prompt that describes the environment and the task (e.g., CTF challenge or delegated task); and (3) the conversation history of agent actions and observations. Following the ReAct strategy, we prompt the LLM to reason and produce an action. We utilize the function calling features of current LLMs to produce actions, so we do not define a structured format of our own, but instead rely on the LLM provider’s API to parse the actions correctly. At every iteration, the conversation history is sent to the LLM and it generates a message containing the reason and action. Observations from executing the actions are appended to the conversation history. The generated reason, action, and corresponding observation constitutes a “round” of conversation. The agent continue these rounds until the task is complete or the context is full. To avoid filling up the context, we truncate observations to 25,000 characters. We also optionally truncate actions and observations in all but the last few rounds while keeping the reasoning intact, similar to Abramovich et al. [1].

B. Environment and Tools

All agents interact with the same Linux container environment containing the CTF files and providing network access to the CTF server and the internet to install new packages. The agents have access to the following tools: RunCommand to execute shell commands; CreateFile to create a file; Disassemble and Decompile to trigger Ghidra¹ to reverse engineer a binary; SubmitFlag to submit a CTF flag to solve the challenge; and, Giveup to giveup solving. Unlike EnIGMA [1], we do not implement advanced interfaces or interactive tools. The specialized reverse engineering tools offer the agents access to Ghidra which does not provide a direct command line interface. We also provide special actions for interaction between agents: GeneratePrompt for the Auto-prompter to generate a prompt; Delegate for the Planner to delegate tasks; and, FinishTask for the Executor to terminate and return a task summary.

C. Workflow

Auto-prompter

Autonomously generated after several exploratory turns

Challenge: Collision Course
Objective: Recover the administrator's password that was encrypted using original database IDs. The password is stored in password.bin and was encrypted using a custom AES implementation.
Files Analysis: encrypt_database.py is the main script that processes the database and encrypts the password...
Observations: Each ID was hashed using MD5 with a 3-character salt... only first 4 characters are used... The encryption password is created by concatenating all original IDs...
Attack Strategy: Since we know original IDs were 1-500..., we can brute force the salt by trying all possible 3-char combinations, find the salt..., use the mapping to recover..., and use my_aes.py to decrypt password.bin.

Hard-coded prompt template

IMPORTANT TIPS:
 You are an autonomous agent... The docker environment already has 'gmpy2' and 'sagemath' installed...
Description: The CTF challenge is a crypto problem named "Collision Course", worth 500 points. A database administrator wrote a script to create unique IDs from the original numeric IDs contained within a database. While doing so, they decided to use the original IDs to encrypt their password, since they were sure the original IDs couldn't be recovered. Prove the administrator wrong and recover the password.
Files included: ['handout.zip']

Generate a step-by-step plan to solve the challenge, then delegate each step to the Executor along with a detailed task description using the 'delegate' tool.

Fig. 3. Auto-prompter generated prompt vs. hard-coded template for the collision_course CTF. Auto-prompter's dynamic prompt captures the approach tailored for this CTF.

1) *Auto-prompter:* The Auto-prompter agent is provided the CTF information in it's initial prompt. Instead of directly rewriting this prompt like typical auto-prompting applications, the agent first interacts with the CTF environment for a few rounds by running commands to read available files, execute the CTF binary, or access the CTF server. Based on this exploration, it generates a prompt tailored for this CTF by calling GeneratePrompt. Figure 3 shows an example auto-prompt and the hard-coded prompt for the collision_course cryptography CTF from NYU CTF Bench. Along with a relevant description, the Auto-prompter proposes a viable approach based on it's exploration. The hard-coded templates provide generic directions that cannot be tailored to each CTF.

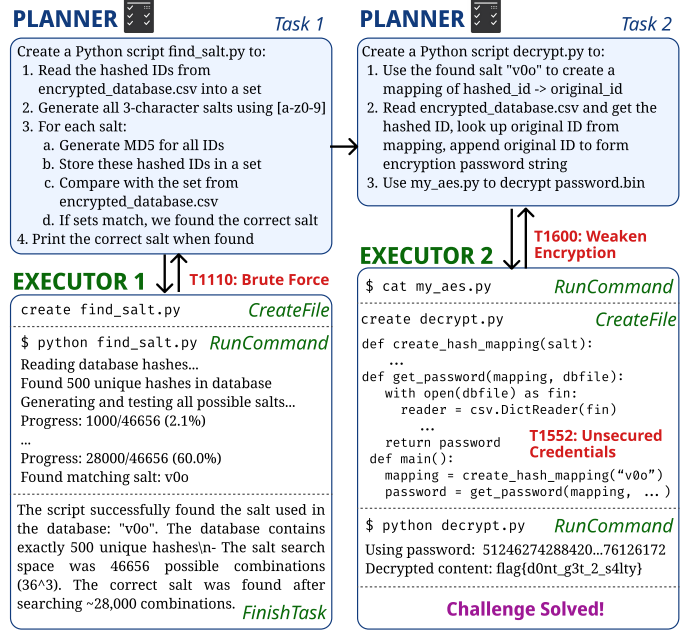


Fig. 4. Planner and Executors interact for the collision_course cryptography CTF. Planner drives the problem solving, while each Executor focuses on delegated tasks and implements specific MITRE ATT&CKs.

2) *Planner-Executor system:* The Planner is initiated with the generated prompt and also explores the CTF for a few turns. It is provided with RunCommand but not CreateFile, Disassemble, or Decompile, allowing exploration, but dissuading it from trying to solve the CTF by itself. It comes up with a step-by-step plan and delegates tasks to an Executor by calling Delegate. The Executor is initiated with the task details and performs the task by running commands and creating scripts, after which it calls FinishTask with a task execution summary and results. The summary is returned to the Planner as an observation, using which the Planner continues to revise its plan and delegate further tasks. For each Delegate call, the framework initiates a new Executor with a new conversation history. Effectively, D-CIPHER runs multiple heterogeneous Executors to solve one challenge. Each Executor focuses on it's own task, while the Planner only sees the task summary, allowing for efficient context management. Figure 4 shows an example of the Planner solving the collision_course CTF. Based on the Auto-prompter's suggested approach and it's own exploration, the Planner starts by delegating the task of cracking the hash salt using brute force. The first Executor successfully implements the brute force attack, correctly employing the T1110 (Brute Force) ATT&CK technique, and returns the task summary along with the hash salt to the Planner. The Planner then reasons and delegates the next step to use the salt and decrypt the password. The second Executor implements the decryption script, employing the two techniques T1600 (Weaken Encryption) and T1552 (Unsecured Credentials). Successfully executing the script reveals the flag and solves the CTF. This example shows how the Planner focuses on the entire CTF while each Executor focuses on single tasks. The workflow ensures continuous interaction between the Planner

¹Ghidra is a popular reverse engineering tool. <https://ghidra-sre.org/>.

and Executors such that they collaborate to enhance problem-solving. Enhanced focus on single tasks also improves D-CIPHER’s capabilities on MITRE ATT&CK techniques.

For each agent, we define a maximum number of conversation rounds after which the agent is stopped. We also set a maximum cost limit of all agents. Among all agents, only the Planner has access to `SubmitFlag` or `Giveup` tools, making it the central agent of D-CIPHER. D-CIPHER terminates when `SubmitFlag` is called with the correct flag, `Giveup` is called, the Planner exhausts its rounds, or the cost limit is reached. If a wrong flag is submitted, a negative response is returned and solving may continue. The Auto-prompter and Executor sometimes exhaust their conversation rounds only running commands and fail to produce an output for the Planner by calling `GeneratePrompt` or `FinishTask`. In that case, we prompt the agents one last time and insist on producing an output. If the Auto-prompter fails, a hard-coded prompt is used. If the Executor fails, a hard-coded warning is returned. To improve focus and avoid exceeding the LLM context, we truncate the Executor’s conversation history to include only recent actions and observations.

IV. EXPERIMENT SETUP

Each run of D-CIPHER attempts one CTF challenge. D-CIPHER is configured as follows: a total cost limit of \$3, a temperature of 1.0 for each LLM, 5 max rounds for the Auto-prompter, 30 max rounds for the Planner, 100 max rounds for each Executor, and each Executor’s conversation history is truncated to last 5 actions and observations.

A. Benchmarks

We evaluate D-CIPHER on NYU CTF Bench [31], Cybench [49], and HackTheBox [17]. As shown in Table II, these benchmarks have 290 CTFs spanning six categories: cryptography (crypto), forensics, binary exploitation (pwn), reverse engineering (rev), web, and miscellaneous (misc). We perform ablation studies on NYU CTF Bench. During development, we use the development set of 55 CTFs introduced in Abramovich et al. [1]. We use the unguided mode of Cybench that does not include additional subtask information with the “hard prompt” that does not contain extra hints.

TABLE II
BENCHMARKS FOR EVALUATING D-CIPHER.

	crypto	foren	pwn	rev	web	misc	Total
NYU CTF	53	15	38	51	19	24	200
Cybench	16	4	2	6	8	4	40
HackTheBox	30	0	0	20	0	0	50
Total	99	19	40	77	27	28	290

B. LLM Selection

For our experiments, we use the same LLM for all three agents. We access LLMs via their APIs. Open-source LLaMa models are accessed via the Together AI platform².

We use the following LLMs: Claude 3.5 Sonnet (*claude-3-5-sonnet-20241022*), GPT 4 Turbo (*gpt-4-turbo-2024-04-09*), GPT 4o (*gpt-4o-2024-11-20*), LLaMa 3.1 405B (*meta-llama/Meta-Llama-3.1-405B-Instruct-Turbo*), and Gemini 1.5 Flash (*gemini-1.5-flash*).

D-CIPHER supports different LLMs for each agent. We explore configurations that pair stronger models for the Planner with weaker models for the Executor. The weaker LLMs used include Claude 3.5 Haiku (*claude-3-5-haiku-20241022*), GPT-4o Mini (*gpt-4o-mini-2024-07-18*), LLaMa 3.3 70B (*meta-llama/Llama-3.3-70B-Instruct-Turbo*), and Gemini 1.5 Flash 8B (*gemini-1.5-flash-8b*). GPT 4o Mini (*gpt-4o-mini-2024-07-18*), LLaMa 3.3 70B (*meta-llama/Llama-3.3-70B-Instruct-Turbo*), and Gemini 1.5 Flash 8B (*gemini-1.5-flash-8b*).

C. Evaluation Metrics

The primary evaluation metric is percentage of CTFs successfully solved (*% solved*). A CTF is solved when the correct flag is submitted by the Planner or if the correct flag is observed in the agent conversation. The latter prevents failures where the Auto-prompter or Executors find the flag but do not tell the Planner, as only the Planner can submit a flag. False positives are highly unlikely because flags are unique strings with specific formats such as `flag{...}`. We also measure the average cost of solved CTFs (*\$ cost*). The total cost of one CTF is the US dollar cost of all LLM API calls across agents. The API cost is indicative of the computational resources required to deploy LLMs, so this metric estimates computational resources for solved CTFs.

D. MITRE ATT&CK Classification

The MITRE ATT&CK framework [24] is a popular taxonomy of offensive security tactics, techniques and procedures used to classify cyber attacks [8]. CTF challenges emulate real-world cyberattack scenarios and we can attribute a CTF to a list of ATT&CK techniques that must be employed to solve that CTF. For all the CTFs that an agent solves, we aggregate each CTF’s mapped techniques that the agent successfully employs to comprehensively benchmark its offensive security capability using the ATT&CK framework.

To perform this analysis, we manually labeled the 200 CTFs in NYU CTF Bench with ATT&CK enterprise techniques as a part of this work. We performed the labeling based on the CTF description, solution writeups and scripts, and manual interaction with the CTF. We then mapped the ATT&CK techniques that must be employed to solve the CTF. Some CTFs only test specific skills and do not involve any attack, especially in cryptography, reverse engineering and miscellaneous categories, so 83 of the 200 CTFs have no techniques that apply. On the remaining 117 CTFs, we mapped 211 instances of 45 unique techniques. Table VII in Section V-G shows the list of techniques and number of CTFs that they apply to. The frequently occurring techniques such as T1600 (Weaken Encryption) and T1552 (Unsecured Credentials) apply to many cryptography CTFs, while T1203 (Exploitation for Client Execution) and T1574 (Hijack Execution Flow) apply to binary exploitation CTFs. Performance on these techniques

²<https://www.together.ai>